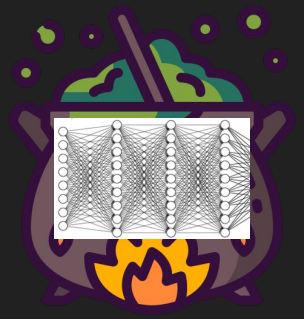


21st Century Alchemy

Part 3: Attention and Transformers

Nicholas J. Cooper

Table of Contents

Last Time…

- Recognizing moving patterns with convolutional layers
  - Fully connected networks lack translation equivariance, CNNs have this property
- Activation functions and vanishing gradients
  - Any small term in backprop eqs. can kill parameter updates → learning stops
- Residual networks
  - Skip connections — parallel paths from input to output
  - Improve gradient flow in very deep networks → better learning

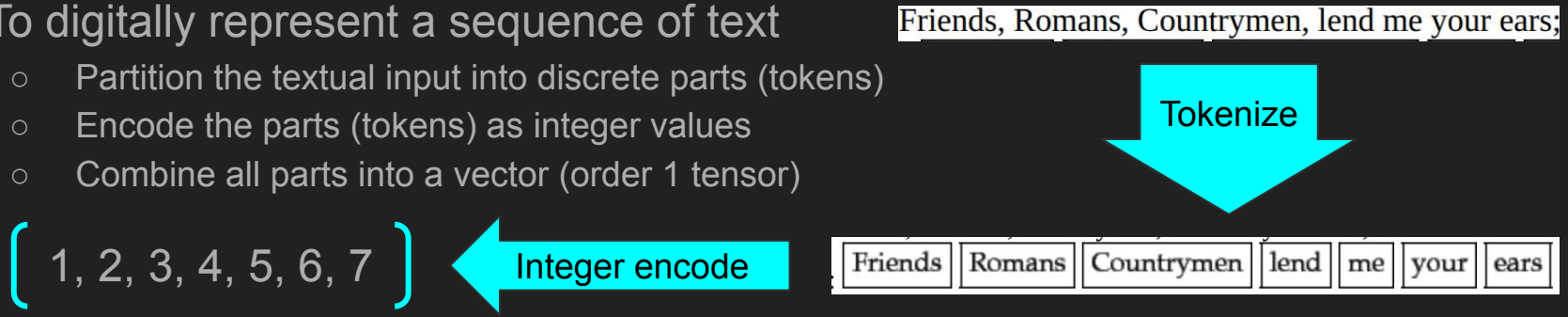
This Time…

- We will see that recurrent networks (RNNs), precursors of transformers, are *dual* to residual networks in the following sense:
  - Skip connections — parallel paths from *input* to output
  - Recurrent layers — parallel paths from *weights* to output
- Despite their parameter skip connections, RNNs are limited in the depth of context they can process effectively
  - Enter the attention operation
- How to build your transformer

Recurrent Neural Networks

Representing Language Digitally

- Recall: to digitally represent an image
  - Partition the visual input into discrete parts (pixels)
  - Encode the parts (pixels) as integer values
  - Combine all parts into an order 3 tensor
- To digitally represent a sequence of text
  - Partition the textual input into discrete parts (tokens)
  - Encode the parts (tokens) as integer values
  - Combine all parts into a vector (order 1 tensor)

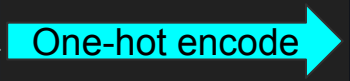


Picture credits: https://nlp.stanford.edu/IR-book/html/htmledition/tokenization-1.html

Representing Language Digitally

- Typically, these ***sequences of tokens*** are one-hot encoded for use in DNNs
  - Useful so models can accept vectors of vectors (matrices) as input
- The collection of all possible tokens is called the ***vocabulary***
- For language modeling, the output is the same as in multiclass classification
  - That is, a vector of size |vocabulary|

Label encoding	
Food Name	**Categorical #**
Apple	1
Chicken	2
Broccoli	3

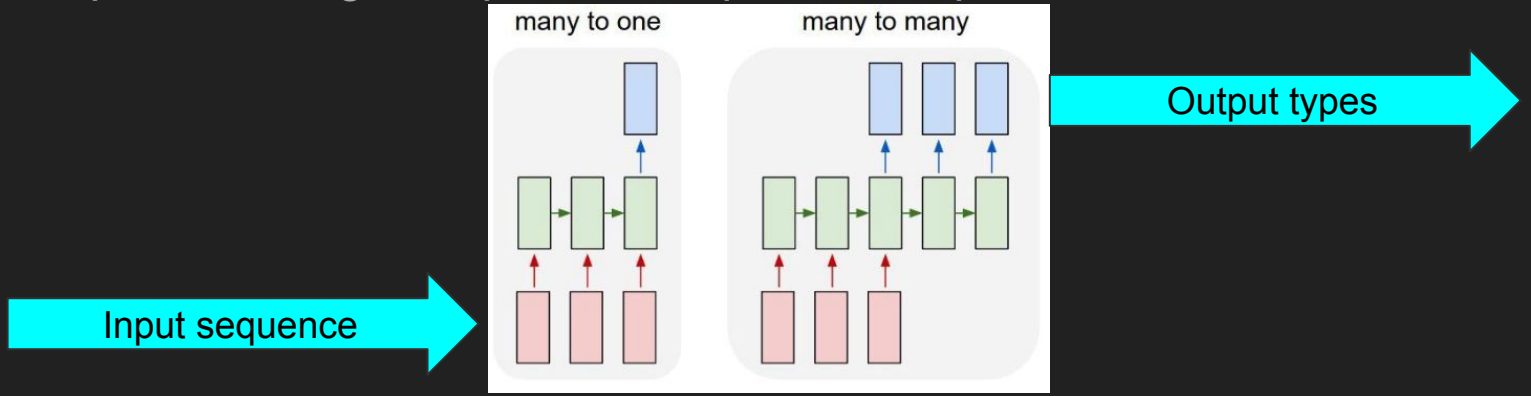


One-Hot encoding		
Apple	**Chicken**	**Broccoli**
1	0	0
0	1	0
0	0	1

Picture credits: Wikipedia

What are RNNs?

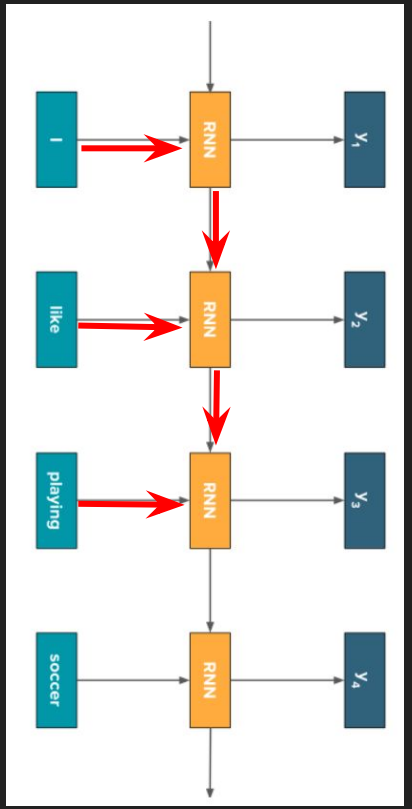
- Neural networks used primarily for natural language tasks, e.g., machine translation
- Accept as input sequence data, such as the words in a sentence
- Can produce single outputs, or sequential outputs



Picture credits: https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf

A Closer Look

- Effectively, each element of the input sequence is fed into the same fully connected layer
- Moreover, the activated outputs from previous instances of the sequence are fed into a second fully connected layer
  - This allows information to propagate along the sequence
- Finally, a third fully connected layer is used to produce the output sequence

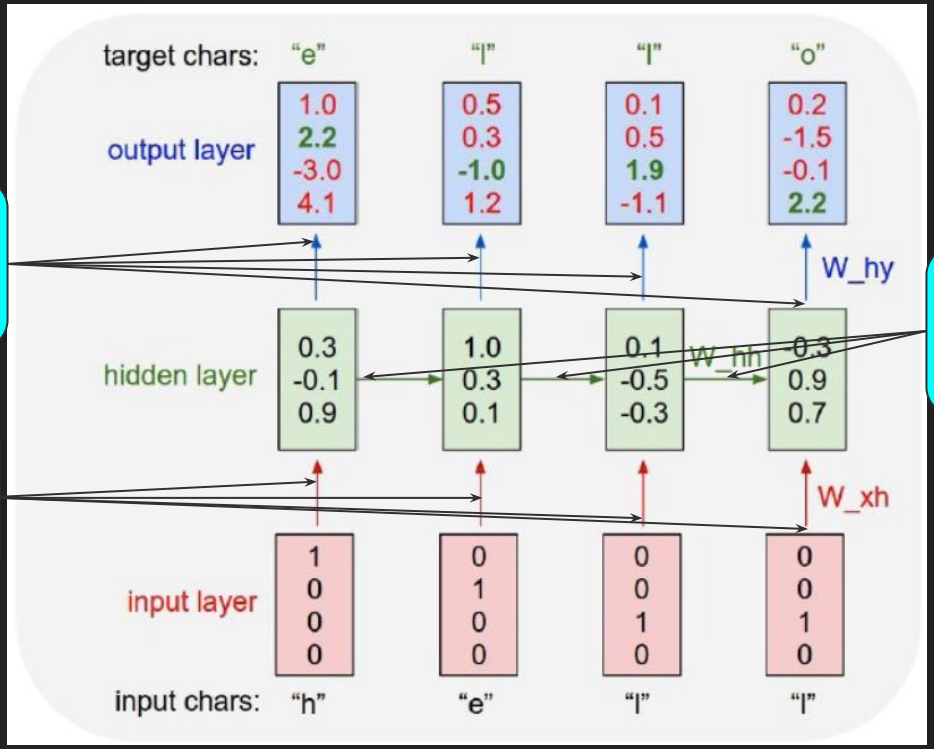


Picture credits: https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf

How do RNNs work?

Weights for the output fully connected layer

Weights for the input fully connected layer

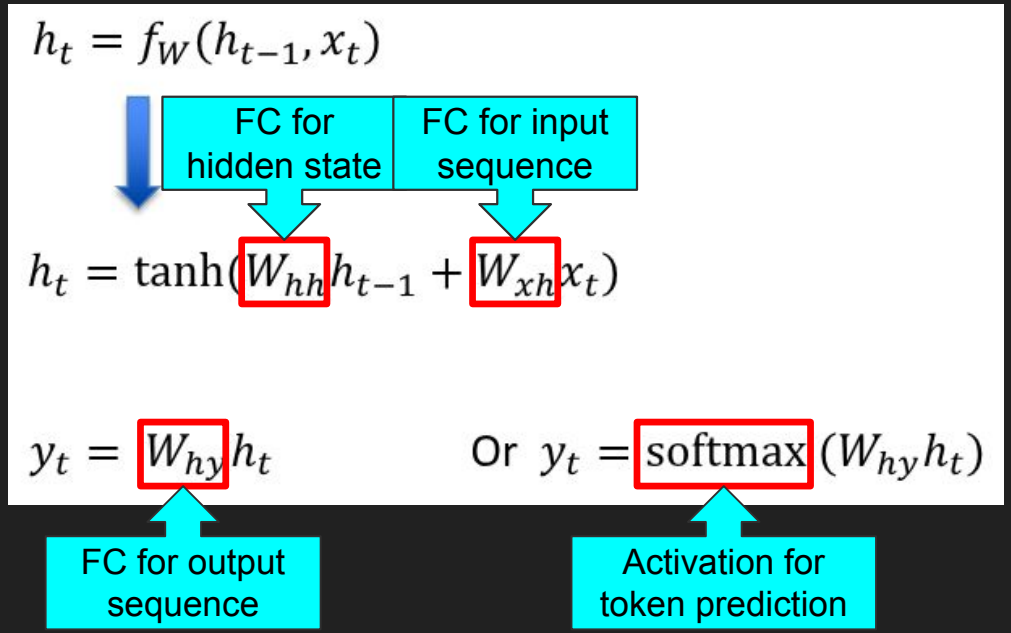


Weights for the hidden state fully connected layer

Picture credits: https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf

How do RNNs work?

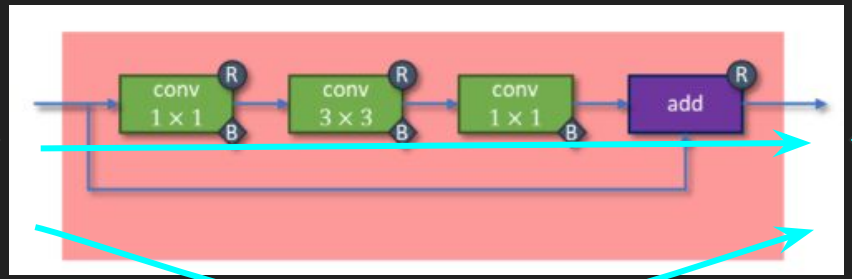
- Forward pass defined by a recursive system of equations



Equation credits: https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf

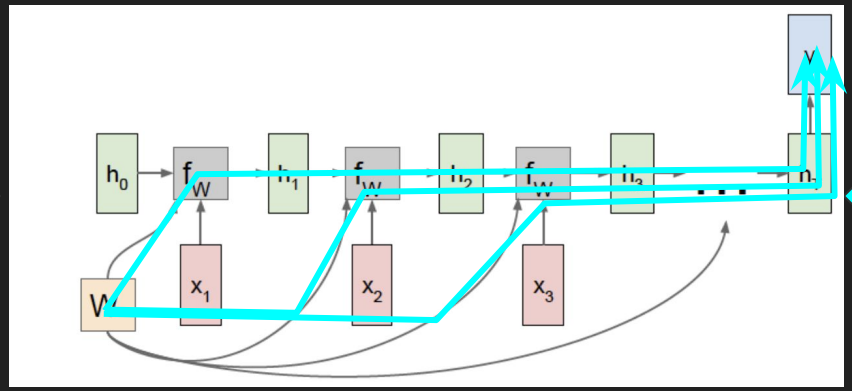
The ResNet ⇔ RNN Duality

ResNet



There are multiple paths through the computational graph from input to output

RNN



There are multiple paths through the computational graph from the parameters W to the output

Picture credits: https://towardsdatascience.com/5-most-well-known-cnn-architectures-visualized-af76f1f0065e/, https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf

RNNs are useful!

Our model	Ulrich UNK , membre du conseil d' administration du constructeur automobile Audi , affirme qu' il s' agit d' une pratique courante depuis des années pour que les téléphones portables puissent être collectés avant les réunions du conseil d' administration afin qu' ils ne soient pas utilisés comme appareils d' écoute à distance .
Truth	Ulrich Hackenberg , membre du conseil d' administration du constructeur automobile Audi , déclare que la collecte des téléphones portables avant les réunions du conseil , afin qu' ils ne puissent pas être utilisés comme appareils d' écoute à distance , est une pratique courante depuis des années .

Ground truth (English): *Ulrich Hackenberg, a member of the board of directors of the car manufacturer Audi, state[illegible]ones before board meetings—to prevent them from being used as remote listening devices—has been common practice for years.*

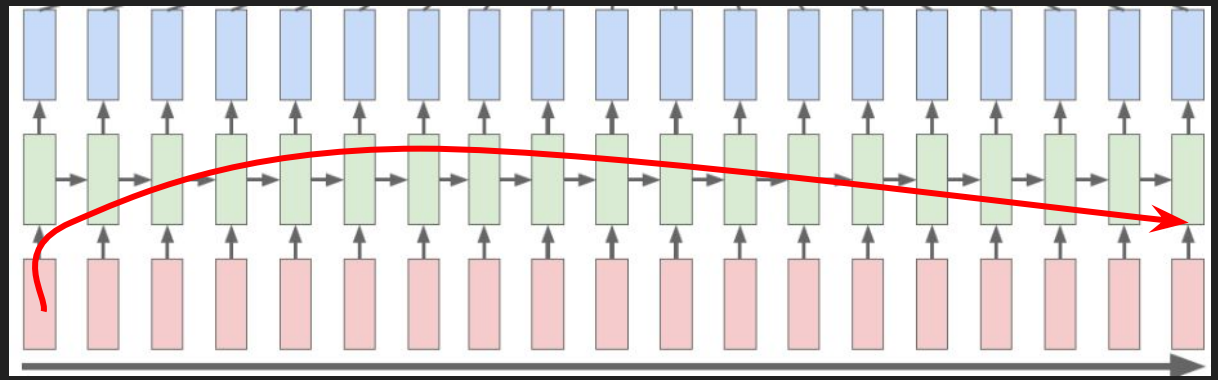
Unknown token, missing from model's vocabulary

Model's output (English): *Ulrich UNK, a member of the board of directors of the car manufacturer Audi, states that it has been common practice for years to collect mobile phones before board meetings to prevent them from being used as remote listening devices.*

Picture credits: Sutskever et. al., *Sequence to Sequence Learning with Neural Networks,* NeurIPS, 2014.

But, there's a problem with RNNs…

- RNNs enjoyed much popularity as the state-of-art approach for machine translation during the early 2010s
- However, they suffer from a "vanishing context" problem
  - By the time earlier elements from the input sequence reach later elements of the output sequence, much information is lost!

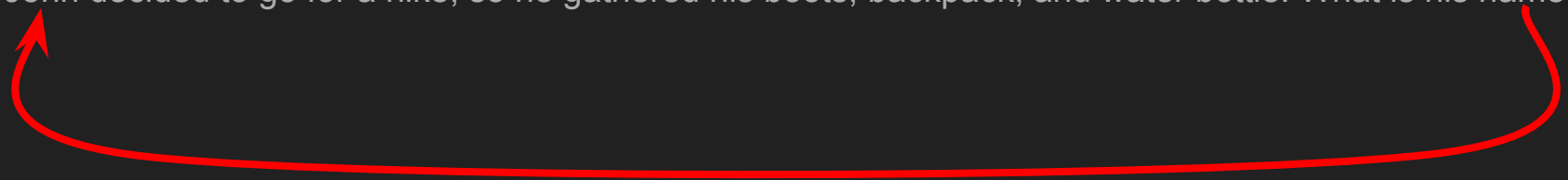


Picture credits: https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf

Example

- Consider how you would answer the question below:

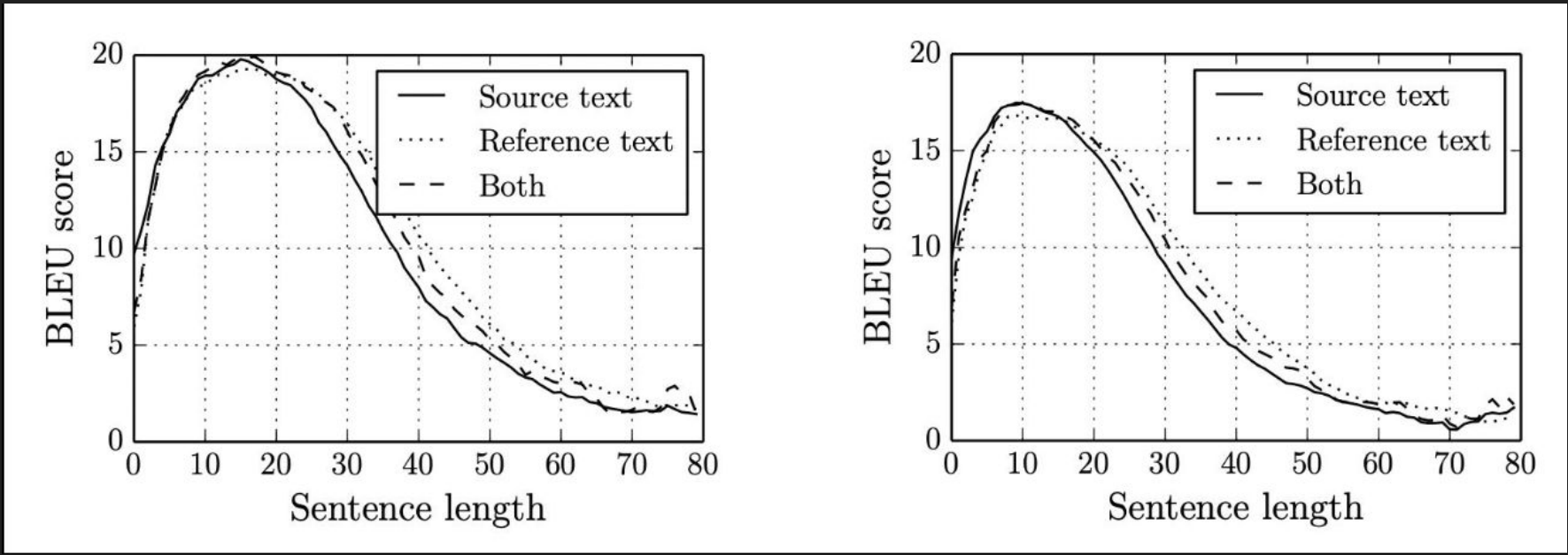
John decided to go for a hike, so he gathered his boots, backpack, and water bottle. What is his name?



- To properly parse this sequence of words, the last element requires information from the first element
  - Context could easily be lost if “John” had to propagate through many recursive computations
- We need “skip connections” for context!

RNNs struggle with long context

- BLEU score (higher better) drops as sentence length increases

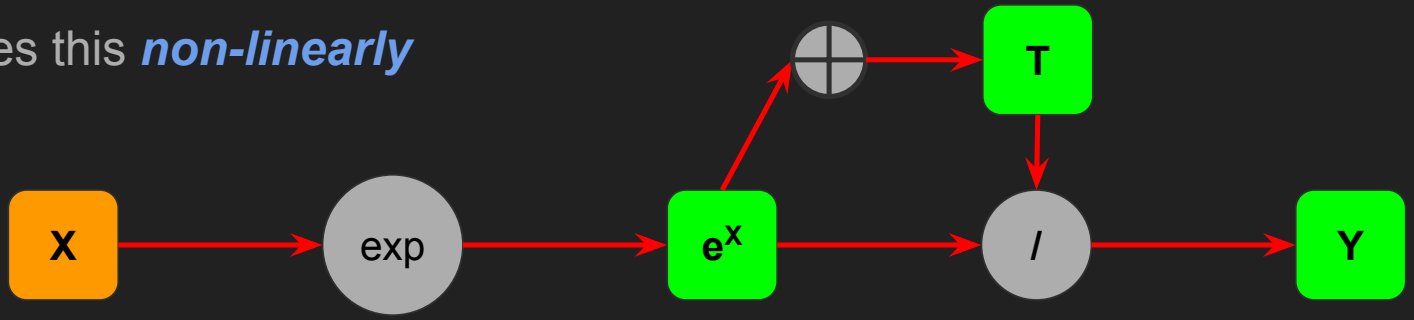


Picture credits: Cho et. al., “On the Properties of Neural Machine Translation: Encoder–Decoder Approaches”, 2014

The Attention Mechanism

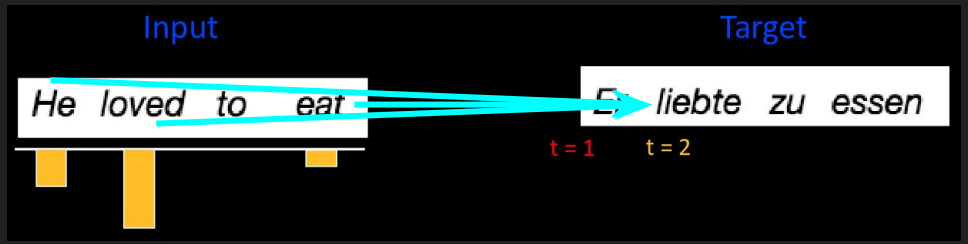
The Softmax Function

- Start with a vector $[x_1, x_2, \ldots, x_n]$
- Send through exp() function $\rightarrow [\exp(x_1), \exp(x_2), \ldots, \exp(x_n)]$
- Sum normalize $\rightarrow [\exp(x_1) / T, \exp(x_2) / T, \ldots, \exp(x_n) / T]$
  - $T := \exp(x_1) + \exp(x_2) + \exp(x_n)$
- Useful for “probablizing” vectors — converting to set of numbers that sum to 1
- Does this ***non-linearly***



Idea: Attention

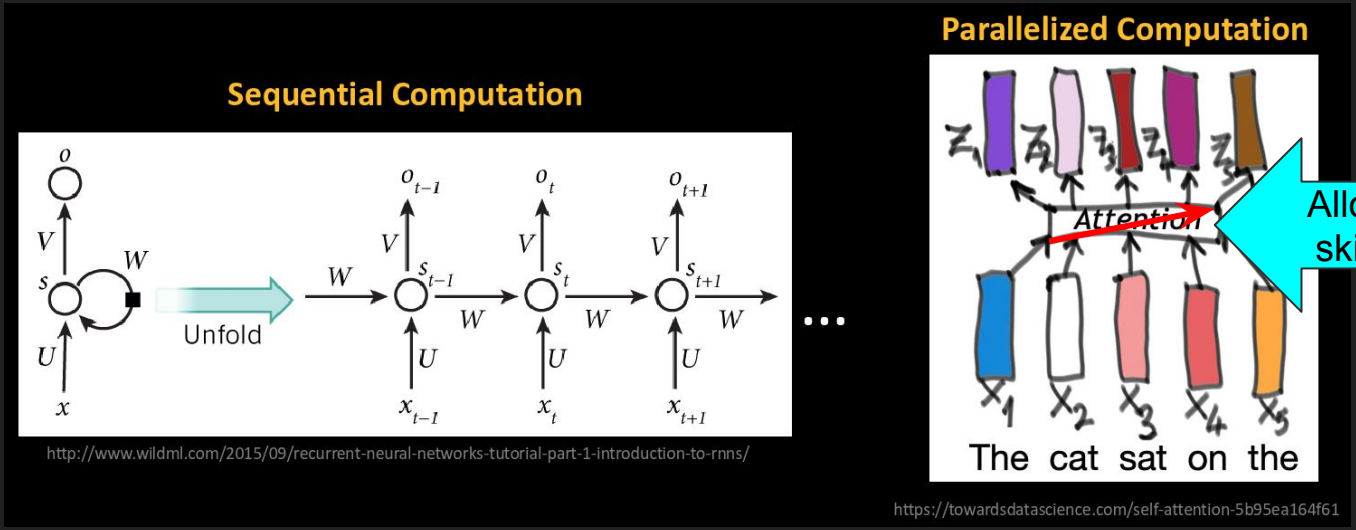
- Instead of recursively propagating each element of the input sequence through a single pathway, introduce “context skip connections”
- “Context skip connections” allow the model to dynamically decide which parts of the input sequence are important to *attend to*
- This is accomplished by forming *weighted sums* of all elements of the input sequence, with different weights for each element of the output sequence



Picture credits: https://home.cs.colorado.edu/~DrG/Courses/2026-Spring-NN/12-Attention.pdf

Attention Intuition

- Recurrent layers process sequential context in *series*
- Attention processes sequential context in *parallel*



Picture credits: https://home.cs.colorado.edu/~DrG/Courses/2026-Spring-NN/12-Attention.pdf

Attention Details

- In a recurrent layer, the output is a function of the hidden state, which integrates all previous sequence elements

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \rightarrow y_t = W_{hy}h_t$$

- In an attention "layer", the output collects a weighted sum of the input

$$\alpha_{t,i} = align(y_t, x_i) = \frac{exp(score(s_{t-1}, h_i))}{\sum_{i'=1}^{n} exp(score(s_{t-1}, h_{i'}))} \rightarrow c_t = \sum_{i=1}^{T_x} \alpha_{t,i}h_i.$$

Attention map values

Softmax of scores for each previous hidden state

Weighted sum of hidden states

Equation credits: https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf, https://slds-lmu.github.io/seminar_nlp_ss20/attention-and-self-attention-for-nlp.html

Attention Details

- There exist many different types of score functions
- Most common is dot-product
- Used in the *self-attention operation*, the core of transformers
- Critically, observe how softmax converts the scores into a *probability distribution* — a set of values that sum to 1
- Allows the output to focus on whichever hidden states are most relevant, regardless of how long ago in the sequence they appeared!

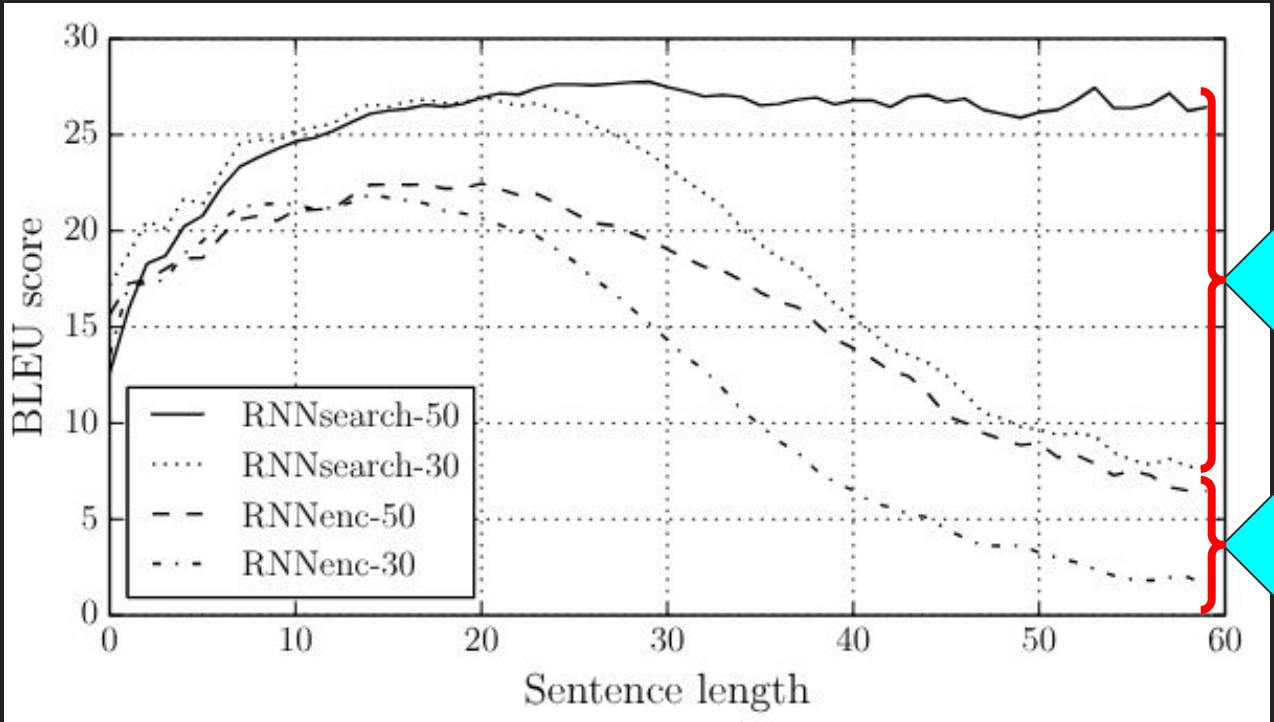
$$\alpha_{t,i} = align(y_t, x_i) = \frac{exp(score(s_{t-1}, h_i))}{\sum_{i'=1}^{n} exp(score(s_{t-1}, h_{i'}))}$$

Equation credits: https://bpb-us-e1.wpmucdn.com/sites.northeastern.edu/dist/f/94/files/2023/05/Math7243Sec12RNN.pdf

Geometry of Attention

- Geometrically, softmax (and other sum-normalization techniques) ensures that the output is a ***convex combination*** of the previous hidden states
  - Output is selected from within the polytope defined by the hidden state vectors
- Softmax pushes output vectors closer towards the polytope's boundary

Attention Works!



Uses attention

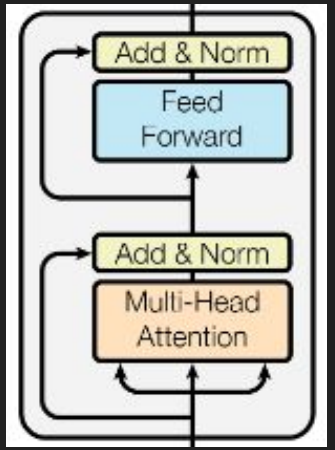
Does not use attention

Picture credits: Bahdanau et. al., “NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE”, 2015

Constructing Transformers

Overview of Construction

- Recall: ResNets are built from residual blocks
- Transformers are built from ***transformer blocks***
- Transformer blocks are themselves built from:
  - A shallow (usually 2 layer) fully connected network (feed forward)
  - A ***multi-head attention block***
  - Several skip connections
- Multi-head attention blocks are built from the ***self-attention operation***
- Self-attention is built from a type of dot-product attention mechanism



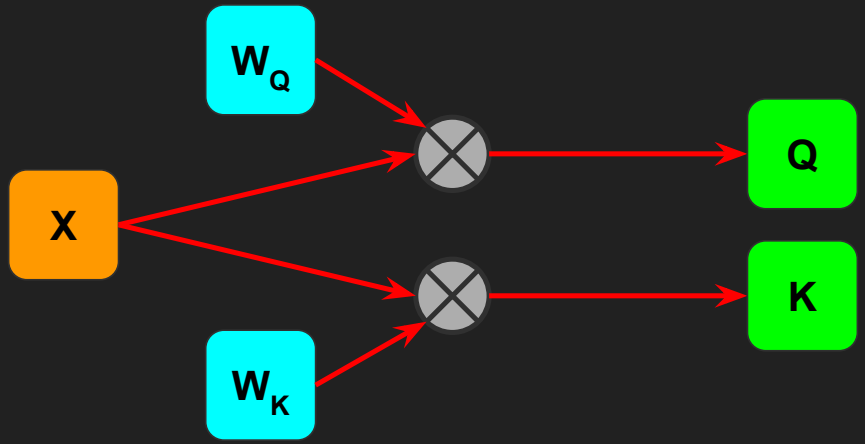
Picture credits: Vaswani et. al., “Attention is all you need”, 2017

The Self Attention Operation

- The first versions of the attention mechanism (from 2014-2015) were used as modifications to RNN architectures
- To construct the transformer architecture, we require self attention layers
  - A particular type of dot-product score attention mechanism
  - Introduced in the foundational 2017 paper “Attention is all you need”
- Throughout, we will assume that the input sequence, X, is stored as a matrix of shape (N, D)
  - N – number of sequence elements or *tokens*; D – dimension of each token

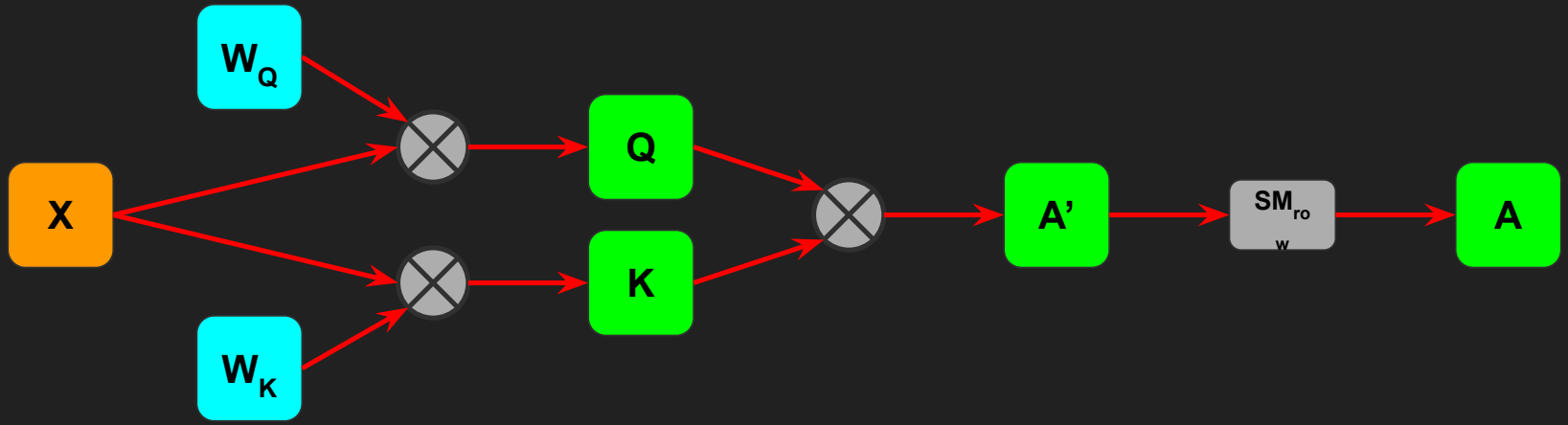
Building Self Attention: Q and K

- Step 1: send the input sequence through two learned fully connected layers
  - $Q = XW_Q$; $K = XW_K$
- Q/K are called the *query* and *key* matrices



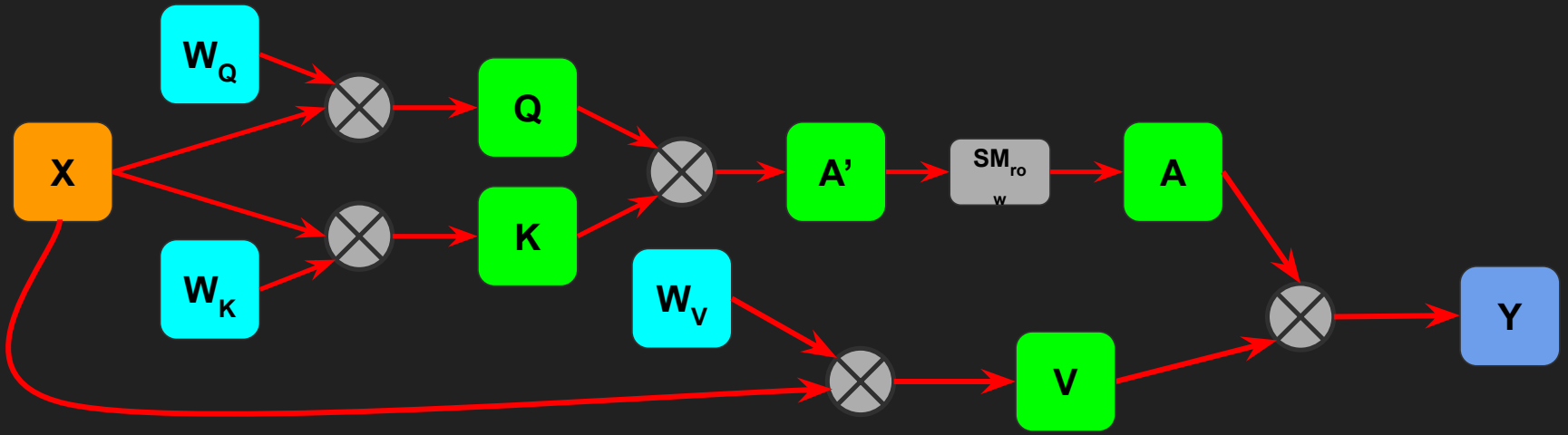
Building Self Attention: The Attention Map

- Step 2: use Q and K to create an *attention map*, then normalize with softmax
  - $A' = QK^T$; $A = \text{softmax}_{row}(A')$
- softmax_{row} is row-wise applied softmax
  - Apply element-wise exp $A''[i,j] = \exp(A'[i,j])$, then $A[i,j] = A''[i,j] / \text{sum}_j(A''[i,j])$

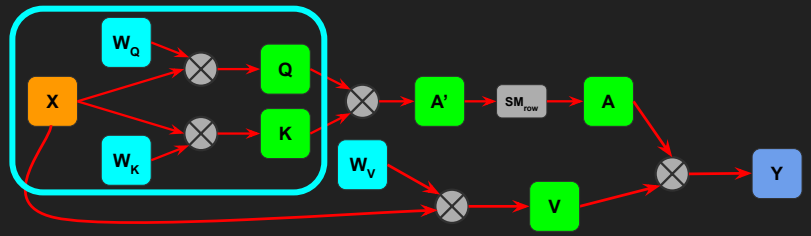


Building Self Attention: V and Y

- Step 3: send the input through a third fully connected layer to produce the *value* matrix V, then form output as convex combinations of V
  - $V = XW_V$; $Y = AV$



Self Attention Intuition: Steps 1-2



D

N

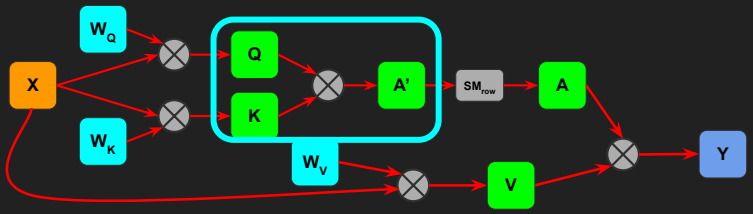
X, contains input tokens

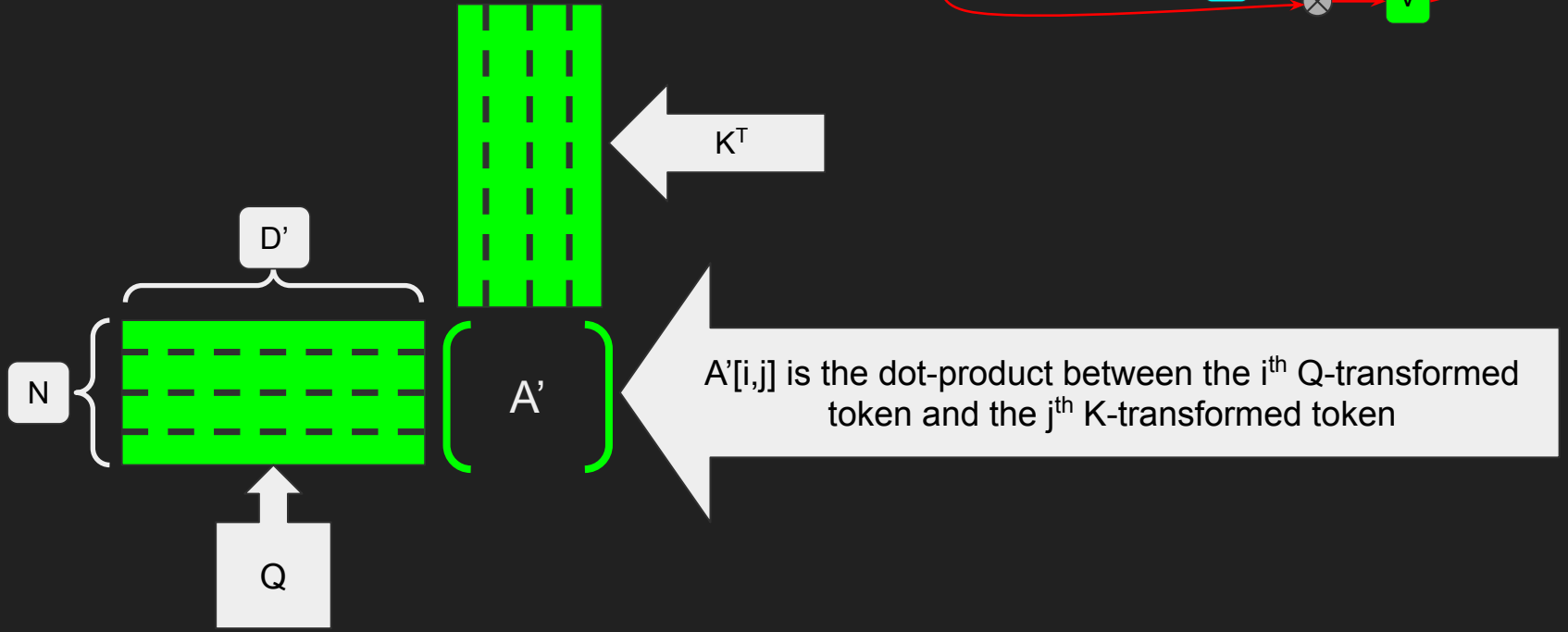
D'

Q, contains transformed tokens

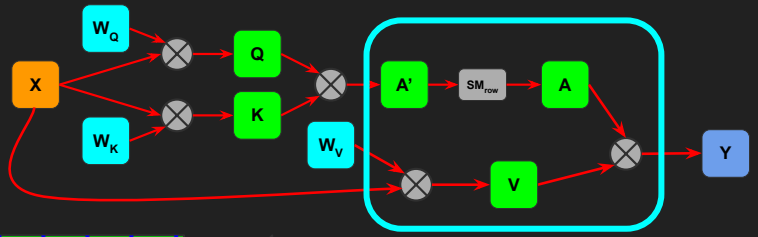
K, contains transformed (indep. of Q) tokens

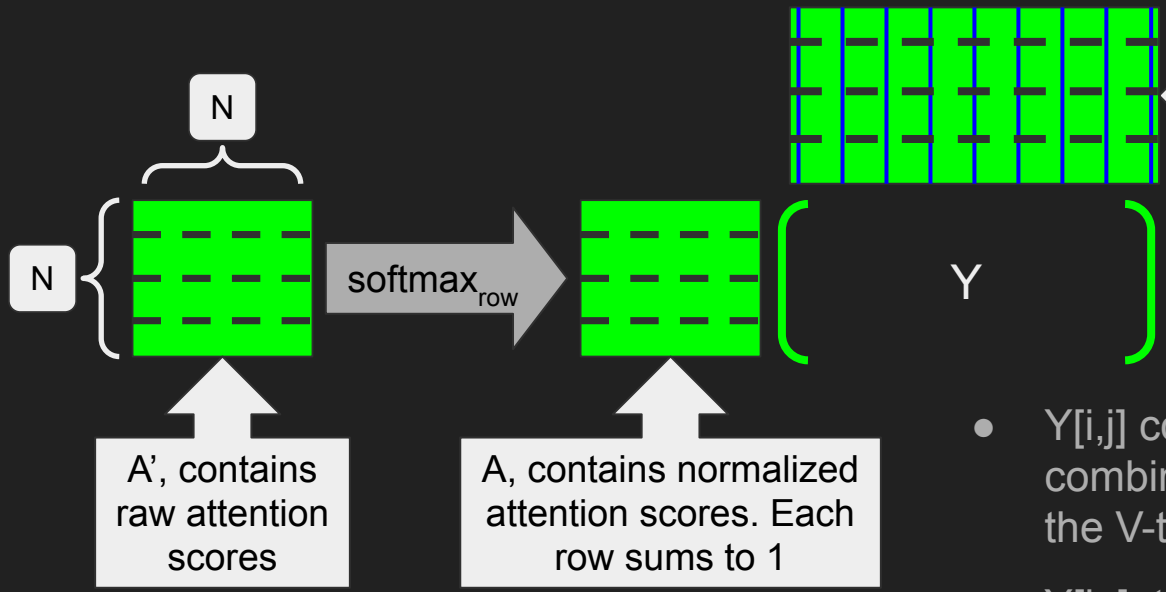
Self Attention Intuition: Steps 1-2





Self Attention Intuition: Step 3





- Y[i,j] contains the i^{th} convex combination of the j^{th} dimension of the V-transformed tokens.
- Y[i,:], the i^{th} row of Y, then contains the i^{th} CC of the V-tokens

Why Query and Key?

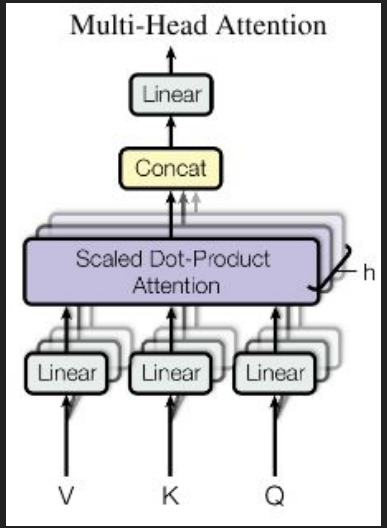
- The i^{th} token of Q and the j^{th} token of K determine A[i,j]
- So, only the the i^{th} token of Q is necessary to determine the i^{th} output token
- However, all N tokens of K are required to determine the i^{th} output token
- Intuitively, the i^{th} “query” is sent to each of the N “keys”
- Queries control differences between the output tokens
- Keys control each dimension of the tokens, across all queries

Attention Visualized

- Demo (code: https://github.com/combinatoriallabs/SEMF_26_Demos/)

Variants of Self Attention

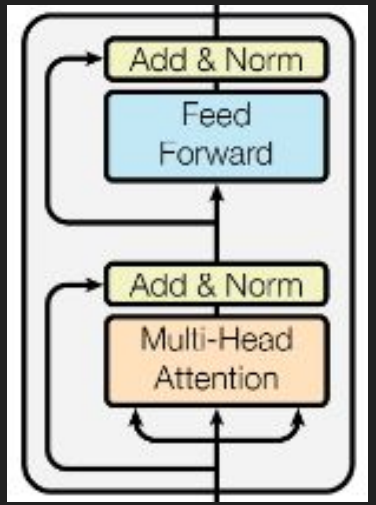
- Cross attention
  - Uses one set of input tokens for Q, and a second set of input tokens for K and V
  - Compare to self attention, which uses the same set of input tokens for Q, K, and V
- Multi-head attention
  - Uses multiple instances of the self attention operation in parallel
  - Outputs are concatenated together
  - Used frequently in transformers



Picture credits: Vaswani et. al., “Attention is all you need”, 2017

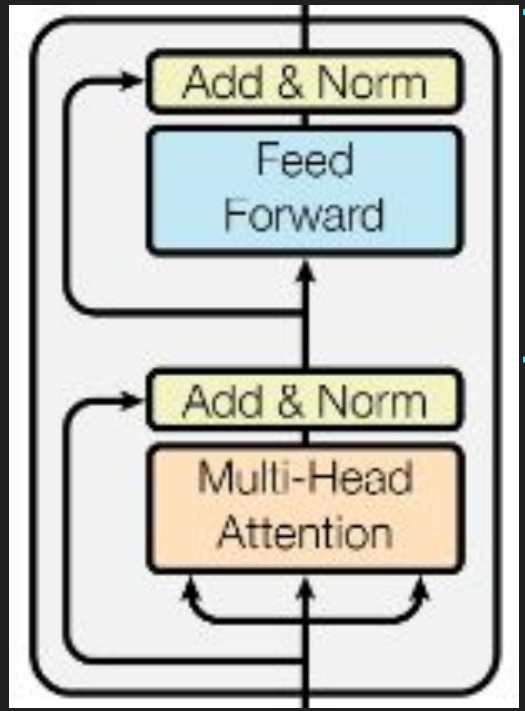
Finally, a Transformer Block!

- We can now construct the transformer blocks used in modern-day LLMs
- While important, attention layers are *insufficient* on their own
  - They lack the expressive power of large, fully connected layers
  - Transformer blocks combine these two layer types
- Pictured here is a *transformer encoder block*
  - Input is passed to multi-head attention
  - Skip connection #1 is sent around the MHA
  - Intermediate result is passed to a feed forward network (fully connected)
  - Skip connection #2 is sent around the FFN



Picture credits: Vaswani et. al., “Attention is all you need”, 2017

Attention is *not* all you need



Higher dimensional internal representations, static input routing

Lower dimensional internal representations, dynamic input routing/context focusing

Picture credits: Vaswani et. al., “Attention is all you need”, 2017

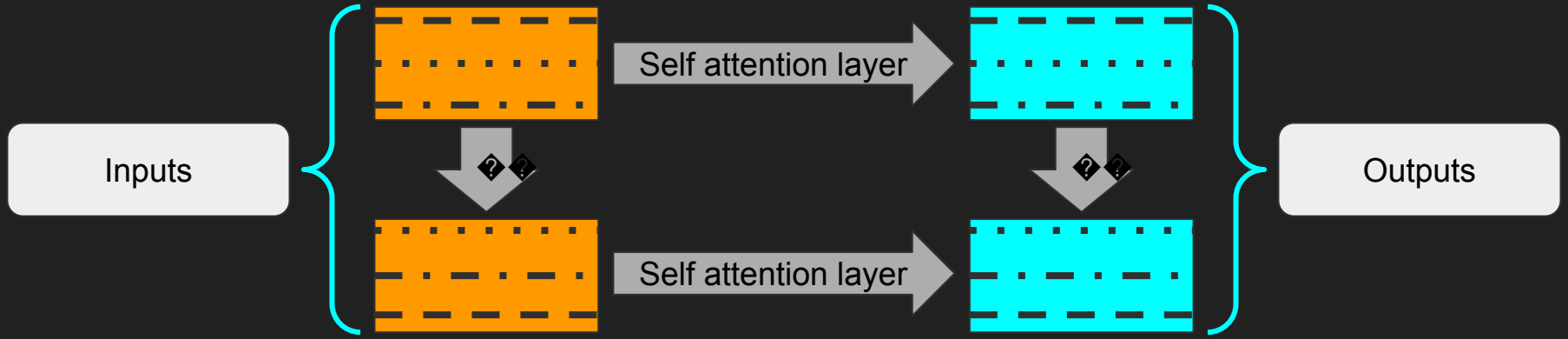
Transformers Visualized

- https://poloclub.github.io/transformer-explainer/

The 2nd Architecture Explosion

Key Benefit of Transformers: Permutation Equivariance

- Similar to how convolutional layers are translation equivariant, self attention layers satisfy a powerful property: *permutation equivariance*
  - Suppose the input tokens are ordered as $(t_1, t_2, \ldots, t_N)$
  - Let $(o_1, o_2, \ldots, o_N)$ denote the corresponding output of a transformer layer
  - If we reorder the input tokens as $(t_{i1}, t_{i2}, \ldots, t_{iN})$, then
  - The corresponding output is reordered in the same way $(o_{i1}, o_{i2}, \ldots, o_{iN})$



Transformers are powerful models

- Permutation equivariance is a formalization of self attention's ability to focus on relevant information regardless of its index in the input token sequence
- This enables transformers to achieve remarkable empirical success
  - They are the de-facto architecture choice for language modelling and LLMs
  - Starting in 2021, they have also revolutionized computer vision
  - Gone on to become ubiquitous components of many architectures

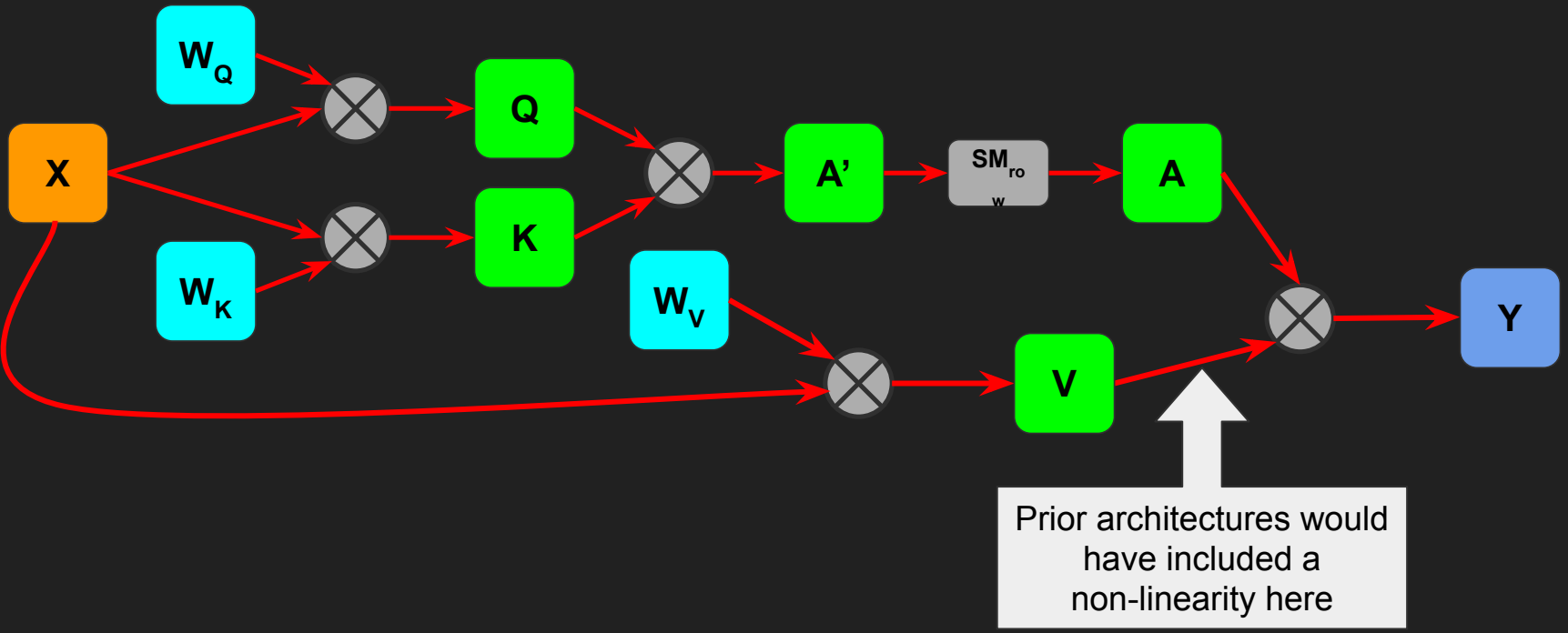
	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70

Picture credits: Dosovitskiy et. al., "An Image Is Worth 16X16 Words: Transformers For Image Recognition at Scale", 2021

Self Attention is a new type of “layer”

- The value tokens, V, are the *unactivated* result from a learned linear transformation
- Yet, A (learned attention map) and V are immediately combined to produce the output Y, *without any activation* in between!
- This marks a significant shift in neural network architecture design
  - Before 2017, *no architecture* utilized sequential linear transformations without some non-linear activation in between

The Missing Activation



A New Explosion of Architectures

- Since the advent of the transformer in 2017, many new diverse types of architectures have been introduced
  - Large language models utilize Transformer encoder/decoder blocks to achieve impressive results on text data
  - Hybrid architectures, e.g., SWIN, combine aspects of CNNs and Transformers to achieve impressive results on image data
  - Polynomial neural networks utilize Hadamard products to expressive higher degree polynomial relationships
  - Tensor tree networks leverage asymmetric interaction modules — combinations of fully connected layers and convolutional operations

What's different this time?

- The 1st architecture explosion was sparked by the introduction of backpropagation
  - Researchers finally had a way to build deep networks
  - More attention was paid to stacking layers, rather than the intra-layer structure
  - Key findings: convolutional networks, recurrent networks, and skip connections
- The 2nd architecture explosion was sparked by a new type of layer
  - Attention layers challenge the linear->activation->linear->activation… status quo
  - Now, much more attention is paid to intra-layer structure
  - Key findings: hybrid architectures, higher order architectures, and likely much much more
- What, if anything, might create the next explosion?

Summary

- Language can be modelled digitally via tokenization
- Recurrent neural networks were the original sequence models
  - Dual to residual networks (parallel paths from parameters to output)
  - Suffer from vanishing context problems
- Attention mechanisms address vanishing context
  - “Skip connections” for tokens
- Self-attention layers are used to build Transformers
  - Fundamentally new type of layer — multiple linear transformations in a row
  - Transformers revolutionized DL — created a 2nd architecture explosion

Coming Up…

- What really “is” generalization?
- Getting a handle on generalization: regularization and inductive bias
- Recent theories of deep learning
  - A whole zoo of them!
- Case studies of “alchemical magic” — deeply surprising empirical results
  - Do we really understand why deep models generalize?